BREAD (FINAL) - INSTALLATION MANUAL (PDF SOURCE)

This markdown file is the source for INSTALLATION_MANUAL.pdf. It is also included in the submission zip for convenience.

SYSTEM REQUIREMENTS

- OS: any OS that supports Docker (Linux/macOS/Windows)
- Docker: 24+ recommended
- Docker Compose: Docker Compose plugin (v2)
- Browser: Chrome/Firefox (latest)
- Disk: enough space for Docker images + volumes (Postgres/MinIO)
- Ports
  - 8080 (default): gateway (frontend + API proxy)
  - 5173 (optional): frontend-dev Vite dev server
  - 15432 (optional): Postgres port mapping for local debugging
  - 15672 (optional): RabbitMQ management UI
  - 9001 (optional): MinIO console

INSTALL DOCKER

If you don’t have Docker installed yet:

1) Install Docker for your OS
   - Docker Desktop (recommended on Windows/macOS): follow the official guide: https://docs.docker.com/desktop/
   - Linux: install the Docker Engine + Docker Compose plugin (v2) following the official guide: https://docs.docker.com/engine/ and https://docs.docker.com/compose/

2) Start the Docker service
   - Docker Desktop usually starts automatically after installation.
   - On Linux you may need to start/enable the daemon (exact command depends on your distro).

3) Verify Docker is working
bash
docker --version
docker compose version

If both commands work, you’re ready to run this project with docker compose up -d.

REPOSITORY CONTENTS (WHAT IS INCLUDED)

The submission zip is generated from this repository and includes the files needed to run the system, excluding:

- .git/
- .env (real secrets)
- node_modules/, .venv/, __pycache__/, build caches, Docker volumes
- dist/ (derived artefacts)

In practice, the zip will include docker-compose.yml, backend/, frontend/, nginx/, tests/, and documentation files like README.md, TESTING.md, and this manual.

ENVIRONMENT SETUP

1) Create your local environment file:

bash
cp .env.example .env

2) Adjust variables in .env if needed.

Required for local Docker run (dev defaults provided in .env.example):
- TZ
- DJANGO_DEBUG
- ALLOWED_HOSTS
- DJANGO_SECRET_KEY
- Postgres: PG_DB, PG_USER, PG_PASSWORD
- RabbitMQ: MQ_USER, MQ_PASS
- MinIO: S3_ACCESS_KEY, S3_SECRET_KEY, S3_BUCKET
- PUBLIC_MEDIA_URL
- GATEWAY_PORT

Optional (can be blank):
- Gmail OAuth variables (GMAIL_*)
- DeepSeek/LLM variables (DEEPSEEK_*)

BUILD & RUN WITH DOCKER COMPOSE

From the project root (same folder as docker-compose.yml):

Build images:

bash
docker compose build

Start the stack:

bash
docker compose up -d

Check status:

bash
docker compose ps
docker compose logs -f

Stop:

bash
docker compose down

To remove data volumes (fresh database / fresh MinIO):

bash
docker compose down -v

DATABASE SETUP (MIGRATIONS / SEED)

- The backend container runs migrations at startup.
- If you need to run migrations manually:

bash
docker compose exec backend python manage.py migrate

HOW TO ACCESS THE SYSTEM

- Main entry (frontend + API gateway): http://localhost:8080 (or http://localhost:${GATEWAY_PORT})
- Backend API (via gateway): http://localhost:8080/api/...

Notes:
- The gateway serves the frontend and proxies /api/ and /media/ to backend/storage.

DEFAULT/DEMO CREDENTIALS

For marking and local testing:
- Verification code is fixed to 1234 in dev/test mode (no real email required).
- Teacher registration invite code comes from TEACHER_REGISTRATION_INVITE_CODE in .env (default in .env.example: 12345678).
- Some flows require .edu.au emails (see tests/integration/README.md for examples).

TESTING INSTRUCTIONS

See TESTING.md for:
- What tests exist (frontend unit, backend unit, integration helpers)
- How to run each test suite
- What must be tested manually (auth, profile, courses, posts, dashboards, uploads, notifications)

Quick commands:

Frontend unit:

bash
cd tests/frontend-unit
npm install
npm test

Backend unit:

bash
python -m venv .venv
source .venv/bin/activate
pip install -r backend/requirements.txt
pip install -r tests/backend-unit/requirements.txt
cd tests/backend-unit
pytest

TROUBLESHOOTING

- Port already in use: change GATEWAY_PORT in .env (and restart: docker compose down && docker compose up -d)
- Containers unhealthy: run docker compose logs -f and verify Postgres healthcheck passes
- Database connection errors: ensure postgres is healthy; check PG_* values in .env
- Uploads not accessible: ensure PUBLIC_MEDIA_URL matches gateway URL and that minio + minio-init are running
- AI features slow or failing: leave DEEPSEEK_API_KEY empty to disable AI, or increase gateway timeout if enabling
- Gmail not configured: leave GMAIL_* blank; demo mode should still allow core flows to work